

Esercizio A1 – Punti ≥ 8

È facile vedere che la funzione è $\Omega(n^3 \log n)$, per via della parte non ricorsiva. Proviamo quindi a vedere se la funzione è anche $O(n^3 \log n)$.

- **Caso base:** per $n = 1$, si ottiene $T(1) = 1 \leq cn^3 \log n = 0$, il che è ovviamente falso. Partiamo quindi da $n = 2$; dobbiamo provare tutti i casi compresi fra 2 e 7, perché solo a partire da $n = 8$ il valore $\lfloor n/4 \rfloor$ è maggiore uguale a 2 (primo caso base). Per $i = 2, \dots, 7$, abbiamo $T(i) = 1 \leq ci^3 \log i$, il che significa che $c \geq \frac{1}{i^3 \log i}$. Tutte queste disequazioni con segno $\geq$ sono rispettate dal valore che si ottiene per $i = 2$, ovvero sia ha che $c \geq \frac{1}{2^3 \log 2} = \frac{1}{8}$.
- **Ipotesi induttiva:** ipotizziamo che per ogni valore k , $2 \leq k < n$: $T(k) \leq k^3 \log k$
- **Passo induttivo:** consideriamo tutti i casi con $n \geq 8$, tali per cui $\lfloor n/2 \rfloor$ e $\lfloor n/4 \rfloor$ sono maggiori o uguali a 2 (il primo caso base); applichiamo la sostituzione e otteniamo:

$$\begin{aligned}
 T(n) &= 7T(\lfloor n/2 \rfloor) + 4T(\lfloor n/4 \rfloor) + n^3 \log n \\
 &\leq 7c\lfloor n/2 \rfloor^3 \log \lfloor n/2 \rfloor + 4c\lfloor n/4 \rfloor^3 + \log \lfloor n/4 \rfloor + n^3 \log n && \text{Sostituzione} \\
 &\leq 7cn^3/8 \log n/2 + 4cn^3/64 \log n/4 + n^3 \log n && \text{Eliminazione } \lfloor \rfloor \\
 &\leq \frac{7}{8}cn^3 \log n + \frac{1}{16}cn^3 \log n + n^3 \log n && \log n/k < \log n, \text{ con } k = 2, 4 \\
 &= \frac{15}{16}cn^3 \log n + n^3 \log n \stackrel{?}{\leq} cn^3 \log n && \text{Passaggio algebrico}
 \end{aligned}$$

L'ultima disequazione è vera per $c \geq 16$. Abbiamo quindi dimostrato che $T(n) = O(n^3 \log n)$, per $m = 2$ e $c = 16$.

Abbiamo quindi dimostrato che $T(n) = O(n^3 \log n)$, per $c = 16$ e $m = 2$.

Esercizio A2 – Profondità minima – Punti ≥ 10

L'algoritmo proposto è una semplice postvisita. L'idea è di calcolare, per ogni nodo t , la distanza minima fra t e un nodo foglia nel sottoalbero radicato in t . Per questo motivo, se il nodo è una foglia, restituiamo 0. Altrimenti, restituiamo la distanza minima fra i figli sinistro/destro e le foglie radicate nei rispettivi sottoalberi, a cui aggiungiamo 1 per contare t . Per gestire la possibilità che un nodo abbia un solo figlio, utilizziamo il valore $+\infty$ per i nodi **nil**, che così non verrà mai selezionato da $\min()$. Il valore restituito nella radice è il valore che cerchiamo.

La complessità dell'algoritmo è pari a quella di una visita di un albero, ovvero $\Theta(n)$.

```

int minDepth(TREE T)
    if T == nil then
        | return  $+\infty$ 
    else if T.left == T.right == nil then
        | return 0
    else
        | return 1 + min(minDepthRec(T.left), minDepthRec(T.right));

```

Questo algoritmo visita tutti i nodi dell'albero. È possibile migliorare l'algoritmo scrivendo una visita in ampiezza che si ferma alla prima foglia. In alcuni casi, questo può comportare un notevole miglioramento, a scapito di un maggior uso della memoria. La complessità nel caso pessimo, tuttavia, non cambia: in

un albero perfetto, la profondità minima è pari all'altezza dell'albero, quindi bisogna comunque visitare metà dell'albero prima di fermarsi, con costo $O(n)$.

```
int minDepth(TREE t)
  int level = 0
  QUEUE Q = Queue()
  Q.enqueue(t)
  Q.enqueue(nil)
  while not Q.isEmpty() do
    TREE u = Q.dequeue()
    if u == nil then
      level = level + 1
      Q.enqueue(nil)
    else
      if u.left == u.right == nil then
        return level
      else
        if u.left != nil then
          Q.enqueue(u.left)
        if u.right != nil then
          Q.enqueue(u.right)
```

In questa soluzione, ho adottato il trucco di inserire **nil** per terminare il livello, che ho trovato nel compito di Dario Maddaloni.

Esercizio A3 – Individua il singolo – Punti ≥ 12

Il problema è risolvibile tramite un algoritmo divide-et-impera che considera il sottovettore $A[i \dots j]$, inizialmente $A[1 \dots n]$. Il caso base occorre quando $i = j$, ovvero c'è un solo elemento, che è quello da restituire.

Nel caso generale, si considera l'elemento centrale m del sottovettore considerato se dispari, o l'elemento precedente se pari. Se l'elemento m ed $m + 1$ sono uguali, significa che prima di m si trovano tutte coppie e quindi il valore cercato si trova a destra, dalla posizione $m + 2$ in poi. Se sono diversi, si trova a sinistra, entro la posizione m . E' possibile verificare che questo approccio funziona correttamente anche con tre elementi.

Volendo fare una dimostrazione di correttezza più approfondita, l'invariante è che il sottovettore che viene passato in input all'algoritmo inizia in una posizione dispari, ha dimensione dispari, e contiene l'elemento singolo. All'inizio, questo è vero perché $i = 1$, n dispari, e per assunzione contiene l'elemento singolo. Ad ogni passo, le tre proprietà vengono rispettate; al termine, essendoci un solo elemento, questo è l'elemento singolo.

```
findSingle(int[] A, int n)
  return fsRec(A, 1, n)
```

```
fsRec(int[] A, int n)
```

```
if i == j then
    return A[i]
else
    int m = (i + j) / 2
    if m mod 2 == 0 then
        m = m - 1
    if A[m] == A[m + 1] then
        return fsRec(A, m + 2, j)
    else
        return fsRec(A, i, m)
```

Essendo una ricerca dicotomica, la complessità è ovviamente $O(\log n)$.

Alcuni commenti:

- A differenza di altri problemi, questo problema dipende molto dalla numerazione dei vettori. Come al solito, la versione presentata qui assume che i vettori inizino dalla posizione 1.
- Esisteva una versione precedente di questo esercizio, che assumeva che i valori fossero ordinati. Come è possibile vedere, il problema è risolubile anche se i valori non sono ordinati. Ma la soluzione data nell'esercizio del gennaio 2018 vale anche per questo esercizio, come molti hanno notato.

Esercizio B1 – Parentesizzazioni – Punti ≥ 8

Il problema si risolve tramite la tecnica del backtrack, con `printPar()` procedura wrapper che crea un vettore S dove memorizzare le possibili parentesizzazioni e chiama la procedura `printParRec()` che prende in input cinque variabili:

- S è il vettore dove vengono memorizzate le parentesizzazioni
- $open$ è il numero di parentesi attualmente aperte nel vettore S , inizialmente 0
- $toOpen$ è il numero di parentesi aperte ancora da inserire, inizialmente n
- $toClose$ è il numero di parentesi chiuse ancora da inserire, inizialmente n
- i è la posizione dove inserire la prossima parentesi, inizialmente 1

La procedura ricorsiva stamperà la parentesizzazione ogni volta che non esistono più parentesi aperte e chiuse da piazzare. In caso contrario, proverà a piazzare una parentesi aperta (possibile solo se il numero di parentesi aperte da piazzare è maggiore di 0) e/o chiusa (possibile se il numero di parentesi chiuse da piazzare è maggiore di 0 e il numero di parentesi attualmente aperte, da chiudere, è maggiore di 0).

Il numero di parentesizzazioni, come abbiamo visto nel problema del prodotto di catena di matrici, è $\Omega(2^n)$ (più precisamente, $\Omega\left(\frac{4^n}{n^{3/2}}\right)$, il costo della stampa è $\Theta(n)$, quindi il costo di questo algoritmo è esponenziale: $\Omega\left(n \cdot \frac{4^n}{n^{3/2}}\right) = \Omega\left(\frac{4^n}{\sqrt{n}}\right)$.

```
printPar(int n)
```

```
ITEM[] S = new ITEM[2 * n]
printParRec(S, 0, n, n, 1)
```
